Frontend Engineering · MVP Specification

Pay with Quai

Non-custodial merchant payments on the Quai network. This handbook tells the frontend engineer exactly what **UI** to build: pages and routes, the payment modal, connect-wallet, QR payments, merchant onboarding, the MVP dashboard, notifications and balances — plus the data each screen consumes. No packaging, no merchant-site integration.

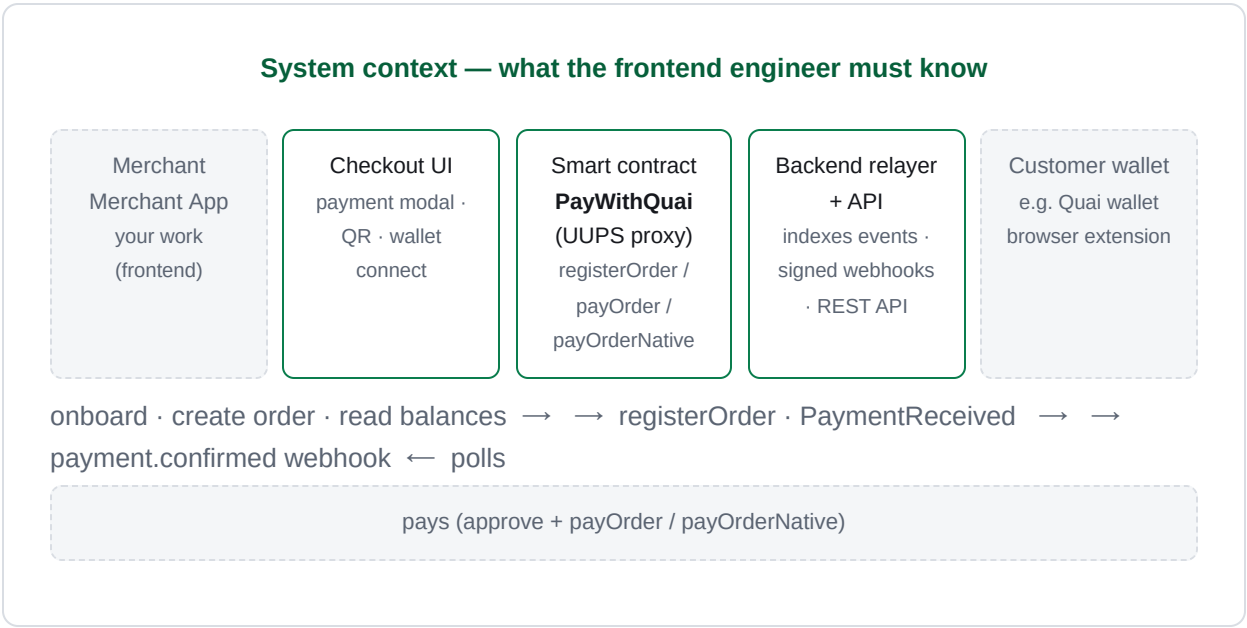
Next.js 16 · App Router | TypeScript | Tailwind v4 | quais SDK | MVP scope

AUDIENCE	REPOS	STATUS	VERSION
Frontend engineers	frontend / contracts / backend	v1 MVP	0.1.0

Contents

01 System overview — how the frontend fits in

The product is **Pay with Quai**: a **non-custodial** payment router on the Quai network. A merchant registers an order on-chain, a customer settles it in one transaction, and the contract verifies the amount, withholds an optional platform fee, forwards funds **directly to the merchant wallet**, and emits a `PaymentReceived` event. An off-chain relayer watches that event and POSTs a signed `payment.confirmed` webhook to the merchant.



Non-custodial by design

Funds never rest in the contract — every payment is routed to the merchant's own wallet in the same transaction. Consequences for UI:

- Balances = on-chain balances of the **merchant wallet**, not a server ledger.
- Payout happens instantly; there is no withdrawal screen in the MVP.
- "Paid" is only true after on-chain finality, not at signature speed.

Single-zone rule

Quai is sharded. The contract, the merchant payout wallet and the payer wallet must all live in the **same zone** as the deployment (e.g. Cyprus-1, addresses starting `0x00`). The UI must warn when a connected wallet is on the wrong zone before showing balance or payment actions.

Key facts that shape the UI

Fact	UI consequence
Everything is keyed to the `PayWithQuai` proxy address (from `contracts/deployments/<network>.json`), zone-scoped (`chainId` like `15000` = Orchard testnet)	Frontend config exposes `NEXT_PUBLIC_CONTRACT_ADDRESS` + `NEXT_PUBLIC_CHAIN_ID`; the app must not hardcode or interpolate the wrong zone.

An order is `(merchant, orderId)`; only the merchant can register/cancel it	The Merchant App creates `orderId` (32-byte hex) and embeds it into the checkout URL/QR payload.
Fee `feeBps` is locked at registration	The UI shows price + fee split *before* the customer pays (amount in, merchant nets `amount − fee`).
Relayer waits for confirmations, re-checks `getOrder().settled`, then webhooks (at-least-once)	Status UI has three macrostates: **pending** → **paid**, plus **failed**. Never show "paid" from a single RPC read without finality awareness.
Backend API is rate-limited (60 req/min/IP) on the public order-status route	Poll politely (5–10 s cadence, back off after 429 with `Retry-After`).

02 Two surfaces: Checkout UI vs Merchant App

There is not one frontend — there are **two surfaces** in the same Next.js app: the **Checkout UI** (the payment-facing components a merchant's customers see) and the **Merchant App** (the dashboard the merchant uses). Build them as separate modules so the checkout UI never depends on dashboard pages (and vice versa).

	Checkout UI	Merchant App (hosted dashboard)
Who uses it	End customers, on the merchant's own site	The merchant team (and, in MVP, internal ops)
Form	React components (`PaymentButton`, `PaymentModal`, `PaymentQR`, `PaymentStatus`) + a hosted checkout page that renders them	Next.js pages under `app/(merchant)/`
Auth	None — public by design. It only builds transactions for the customer's own wallet to sign.	Wallet-based login (sign a message). Admin operations call the backend with the admin bearer key, which the server keeps out of client bundles.
Owns	Wallet connect chip, payment modal states, QR canvas, status polling widget	Onboarding wizard, dashboard, balances, notification center, webhook health, settings
Packaging	Components live in `frontend/src/checkout/` — UI code only, no dashboard imports	The Next.js app itself (`frontend/`)

> **MVP boundary**
>
> Wallet-based auth ("sign-in with Quai") is nice-to-have in the MVP. Minimum viable: the dashboard is behind a simple gate — a dev login that accepts a configured admin key/email — while onboarding (Section 07) already uses the real wallet-connect primitive. Do not build a full auth stack before the payment rails work.

03 Pages & routes to build

Merchant App pages (Next.js App Router)

Route	Page	Purpose
/	Landing	Product intro, "Start accepting payments" CTA → onboarding; "Checkout tools" CTA.
/onboarding	Onboarding wizard	4-step wizard: intro → connect wallet → profile → review & done (Section 07).
/dashboard	Overview	Balance cards, latest payments, live notification feed, webhook health strip (Section 08).
/dashboard/payments	Payments	Table of all payments with status badges, filter (all / pending / paid / failed), webhook delivery detail drawer.
/dashboard/checkout	Checkout tools	Create a payment: amount, token (QUAI / stablecoin), expiry; produces **QR** and a **payment link** ready to share (Sections 06, 07).
/dashboard/settings	Settings	Store name, webhook URL, active toggle (pause/resume delivery), connected wallet display + copy.

Checkout components (rendered on merchant-facing pages and the hosted checkout)

Component	Used in	Purpose
`PaymentButton`	Checkout tools, hosted checkout	Styled trigger that opens the payment modal with the preconfigured order.
`PaymentModal`	Overlay on any page	The checkout: order summary → wallet/zone check → pay → confirmation (Section 04).
`PaymentQR`	Checkout tools, hosted checkout	QR canvas + amount summary + expiry countdown for scan-to-pay (Section 06).
`PaymentStatus`	Payments page, paid screens, notifications	Live order status pill (Pending / Paid / Expired) polling the order-status API.
`openCheckout(order)`	Programmatic	Launches the payment modal from code with an order config.

> **Build a QA demo page**
>
> Keep an internal `/demo` route that walks every component through a real flow (connect → create payment → pay → status → notification), so e2e and manual QA exercise the actual component APIs — never import dashboard state into the checkout components.

04 Payment UI — flow & states

The happy path

1. Order config arrives in the UI (amount, token, orderId, expiry, description) — from checkout tools, a payment link or a page config

customer clicks Pay

2. PaymentModal opens — order summary card (amount, fee split, token, merchant name, expiry)

wallet step

3. Connect wallet / pick wallet + zone check (Section 05)

pay flow per token

4a. Native QUAI → `payOrderNative(merchant, orderId){value}`

4b. ERC-20 → `approve(PayWithQuai, amount)` then `payOrder(merchant, orderId)`

tx sent

5. Awaiting confirmation — spinner, order summary, tx link, "paid" only on final status

relayer confirms

6. Success screen — green check, net amount received by merchant, close → merchant page shows `PaymentStatus` as paid

Modal states (single source of truth: a reducer or XState-like machine)

State	What renders	Transitions
`idle`	Nothing (modal closed)	→ `summary` on open
`summary`	Amount (human units), token badge (QUAI / mUSDQ), fee note "You pay X · Merchant receives Y", merchant name, expiry countdown if set	→ `wallet`; → `error` on bad config
`wallet`	Connect card (Section 05); if wrong zone → blocking warning with "switch" attempt	→ `confirm`
`confirm`	Final breakdown + "Pay {amount} {token}" primary button (disabled while a wallet tx is pending); gas note	→ `processing`
`processing`	Spinner, tx hash link to the zone explorer (Quaiscan), "waiting for confirmation"	→ `success` / `error`
`success`	Big check, amount paid, "Merchant receives {net}", dismiss + optional `onPaid` callback that flips any `PaymentStatus` on the page instantly	→ `idle`

`error`	Inline error banner: user rejected / insufficient balance / order already settled / expired / wrong token. Retry or cancel buttons	→ `confirm` / `idle`

> **Never claim "paid" prematurely**
>
> The modal's success screen may appear when the tx is included, but any persistent UI (`PaymentStatus`, dashboard, notifications) must key off `getOrder().settled` **and** the relayer's webhook delivery (`webhook.status === "delivered"`). The backend re-verifies finality before webhook delivery.

Fees displayed honestly

```
order = await client.getOrder(merchant, orderId)
// { token, amount (gross, smallest units), feeBps, expiry, settled }
fee = BigInt(order.amount) * BigInt(order.feeBps) / 10000n
net = BigInt(order.amount) - fee        // merchant receives net
```

Always format with the token's decimals (native QUAI = 18; stablecoin mock mUSDQ = 6). Show gross, fee and net — the webhook payload later reports the same split (`fee` / `net`), so what the customer saw must match what the merchant counts.

05 Connect-wallet UI & address copy

Discovery & connect card

- **Detect** injected providers via `window.quais` / EIP-6963-style discovery (use the `quais` npm package's wallet helpers — the same SDK the backend uses, so address checksumming and zone derivation match).
- **Render a list** of detected wallets as cards (icon, name, "Most common" tag for the first one), each with `Connect`. Don't auto-connect on page load; **never** pop a provider window without a user gesture.
- **Before doing anything chain-related**, read `ethereum.getChainId()`/quais equivalent and compare with `NEXT_PUBLIC_CHAIN_ID`. The contract zone must match the wallet zone (single-zone rule).
- **Wrong zone** → amber card: "Your wallet is on zone {got}. Payments on this site only work on zone {want}." Offer `wallet_switchEthereumChain`-style switch when the provider supports it; otherwise disable Pay.

Connected state — the address chip (used everywhere: topbar, settings, payment modal)

Chip anatomy: truncated checksummed address `0x{4}…{4}` · copy button with "Copied ✓" micro-feedback (and **persist success for 1500 ms**) · network badge (zone name + chain) · disconnect.

Copy-to-clipboard spec (used for addresses, order IDs, merchant ids, QR links)

Behavior	Requirement
Mechanism	`navigator.clipboard.writeText` with a `document.execCommand` fallback for non-secure contexts; wrap in one `useCopy(opts)` hook.
Feedback	Icon swaps to a check for 1500 ms; button label "Copied"; `aria-live="polite"` announcement.
What is copied	The **checksummed** address (`getAddress()`), never lowercase; the full 32-byte hex `orderId` and merchant ids, never truncated.
Full-form	Clicking the address part opens a small popover with the full address + copy (long addresses are otherwise unusable to check in explorers).
Errors	On permission failure show inline "Copy failed — select manually" and fall back to a readonly input with the text selected.

> **One wallet-store, two surfaces**
>
> Both the checkout UI and the Merchant App need connect-wallet. Implement once in `src/shared/wallet/`, and have both surfaces consume the same primitive (Section 11). The wallet store holds: `provider`, `address`, `chainId`, `zone ok?`, `connecting`, `error`.

06 QR-code payment

QR payload (what the QR encodes)

A single URI the scanning wallet parses. Version the scheme now to avoid a v2 breaking change later:

```
paywithquai://pay?merchant=0x<checksummed addr>&orderId=0x<32-byte hex>
  &chainId=15000&token=0x0000000000000000000000000000000000000000
  &amount=25000000&description=Invoice%20%2377&expiry=1786480000
```

Param	Meaning
`merchant`	Merchant payout address (checksummed).
`orderId`	The registered order id — the wallet should resolve it via `getOrder(merchant, orderId)` and prefer **on-chain truth** for amount/fee; local `amount` acts as a "preferred amount" for display.
`chainId`	Zone guard for the scanning wallet; `15000` = Orchard testnet.
`token`	`0x0…0` = native QUAI, else the ERC-20 address.
`expiry`	Unix seconds; when expired the QR UI shows **Expired** and a "Regenerate" button (merchant must `cancelOrder`/re-register).

Payment QR component (`PaymentQR`)

Region	Content
Canvas	QR rendered from the payload via the `qrcode` package; white background, quiet-zone padding, 220 px; center logo mark optional.
Summary line	**25.00 QUAI** · Invoice #77 · "Scan with your Quai wallet"
Countdown	"Expires in 14:59" ticking down from `expiry`; 5 s before expiry switches to amber, at 0 flips to expired state.
Fallbacks	"Open payment link" (copies the full URL) and "Pay in your wallet" — a deep link a mobile wallet registers for.
Status chip	`PaymentStatus` inline: Pending → Paid flips in place once the relayer confirms; QR self-dims on paid.

> **QR is a renderer, not an order creator**
>
> The backend/relayer is not involved in creating orders; orders exist only after `registerOrder` lands on chain. The Merchant App's **Checkout tools** page must register the order first (sign with the merchant wallet), then render the QR / payment link from the returned state. A QR that points at an unregistered order dead-ends at "order not found".

07 Merchant onboarding (MVP wizard)

Four steps, one file `app/(merchant)/onboarding/`, progress indicator top, "Back" allowed until step 3. Purpose of the wizard: turn a connected wallet into a registered merchant on the backend. The payout address **is** the wallet they connect — the contract pays that address directly.

Step 1 · Intro — "Connect your wallet to begin. Payouts go straight to this address."

Connect wallet (Section 05) · zone check

Step 2 · Profile — store name (≤200 chars), webhook URL (https; validation messages mirror the server)

POST /v1/merchants (admin bearer)

Step 3 · Review & create — summary card (address chip, name, webhook URL) → "Create merchant" button with loading/error states

finish

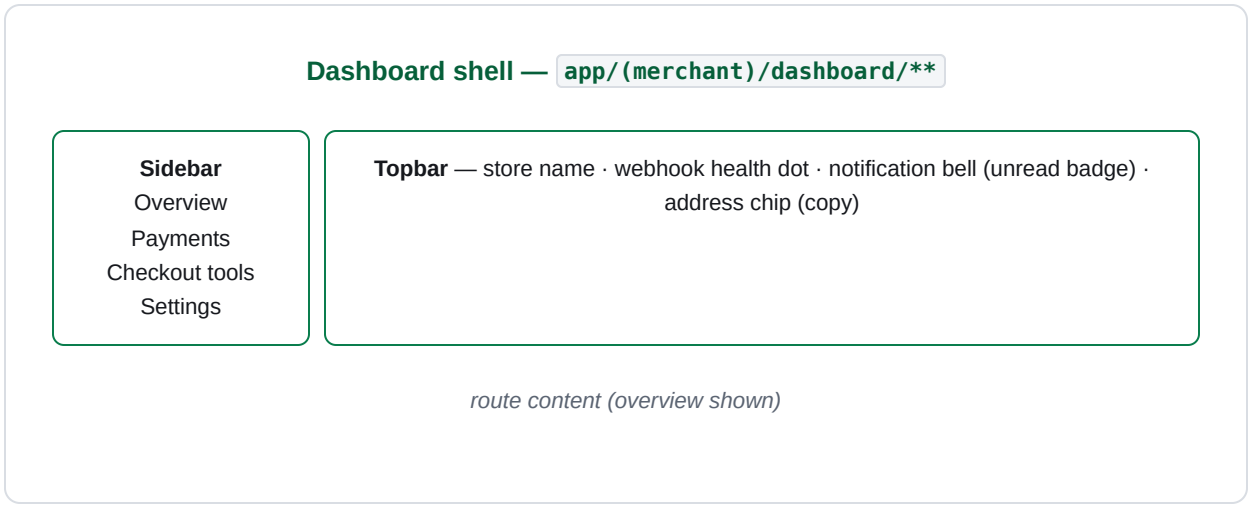
Step 4 · Done — "You're live" → Dashboard; primary CTA "Create your first payment" → Checkout tools

Spec details

Element	Requirement
Admin call	`POST /v1/merchants` with `{ address, name, webhookUrl }` bearer-admin. The dashboard's server-side proxy holds `ADMIN_API_KEY`; the client never sees it.
Duplicate address	`409 merchant already exists` → route to Settings with an inline notice instead of a dead-end error.
Bad webhook URL	Backend SSRF-guard rejects non-https/internal URLs → inline field error mirroring the server message.
Resume	The wizard keeps its state (connected wallet, typed profile) in session storage, so "Back"/"Close" never loses progress.
Done screen	Merchant summary: address chip with copy, merchant id, store name — plus the "Create your first payment" CTA.

08 Merchant dashboard (MVP)

Layout



Browser-native layout, responsive: sidebar collapses to a bottom tab bar ≤ 768 px (mobile is a core merchant use case — QR on a phone).

Overview page (`/dashboard`)

Card / region	Content
Balance cards (Section 10)	Two cards: Native QUAI and stablecoin (mUSDQ). Value in native units + "≈ USD" estimate (hardcoded MVP rate, labeled "approximate"). Each card: full balance, Address chip + copy (payout is direct to this wallet).
Webhook health strip	Dot + text: "Webhook delivering" (green, last OK < 5 min), "Delayed" (amber, retries in progress), "Failing" (red → Settings). From `GET /v1/deliveries` summary.
Latest payments	Last 5-10 rows from the notification feed: amount, token, order id (truncated), status chip, timestamp. "View all" → Payments.
Live feed	Payments arrive appended at top with a subtle highlight animation for ~2 s, driven by the same event bus as notifications (Section 09).

Payments page (`/dashboard/payments`)

Feature	Spec
Table columns	Status · Amount (gross + fee + net tooltip) · Token · Order id (mono, truncated, copy) · Payer (truncated, copy) · Tx hash (links to Quaiscan) · Time.
Status chips	**Pending** (mined, awaiting finality/webhook) · **Paid** (webhook delivered; shows net) · **Failed** (webhook exhausted retries) · **Skipped** (paid to unregistered address — shown under the *All* filter only).
Filters	All / Pending / Paid / Failed; search by order id or payer; MVP: client-side only.

Detail drawer	Full payload mirror of the `payment.confirmed` webhook body (merchant, orderId, payer, token, amount, fee, net, txHash, blockNumber, timestamp) + delivery log (attempts, last error, next attempt) + **Retry** button → `POST /v1/deliveries/:id/retry` (admin).

Checkout tools (`/dashboard/checkout`)

Region	Content
Order form	Amount (human units with token decimal awareness), token select (QUAI / stablecoin), optional expiry (default 30 min), optional description/invoice id.
Register & render	Signs `registerOrder(orderId, token, amount, expiry)` with the merchant wallet; then renders the three artifacts below (Section 06).
QR card	Live `PaymentQR` (Section 06) sized for mobile scanning.
Payment link card	Full URL + copy; opens the hosted checkout page (internal route reusing the payment modal).

> **MVP data source**
>
> No dedicated payments API yet — the dashboard joins three cheap sources: admin `GET /v1/merchants`, admin `GET /v1/deliveries` (webhook history = the payments ledger, keyed `txHash:logIndex`), and `GET /v1/orders/:merchant/:orderId` for live order state. If delivery lookups get heavy, a `GET /v1/payments/:merchant` endpoint is the obvious backend addition — flag it, don't build it yet.

09 Payment notifications & webhook status

Notifications are the moment the merchant learns money arrived. They must be **driven by the same event the ledger uses** (delivery status), so the bell count, the toast and the Payments table can never disagree.

Notification pipeline (MVP)

1. Poll `GET /v1/deliveries` (admin) every 10 s — lightweight, single admin list

new `delivered` id seen

2. Emit `payment.confirmed` on a client event bus (Zustand store + `subscribe`)

bus

3. Side effects: toast · bell badge +1 · latest-payments prepend · mark-as-read on open

A future v2 can swap the poller for SSE/WebSocket server push without touching any consumer.

Toast spec (payment arrived)

✓

Payment confirmed — +25.00 QUAI

Order 0x38c1…d9 — merchant nets 24.87 QUAI · auto-dismiss in 6 s · undo-less (non-destructive)

Rule	Value
Entry	Slide-in bottom-right, stacks, max 4 visible
Content	Type icon, "+{gross} {token}", merchant-nets line, order id link → Payments drawer
Role	`role="status"`, announce via aria-live
Dismiss	Auto 6 s, manual ×, click-through to payments

Notification center (bell in topbar)

Element	Spec
Badge	Unread count; cap display at "99+"; clears when the drawer opens.
Drawer	Last 50 notifications grouped "Today / Earlier"; each row: icon, "Received 25.00 QUAI", net line, time-ago, link to payment.
Kinds (MVP)	**Payment · 25.00 QUAI** (confirmed) and **Webhook failing** (a delivery exhausts retries — amber row with "Review" link to the delivery drawer).

Persistence	Notifications derive from the delivered ledger; the drawer is derived state — no separate storage needed in MVP.

Webhook delivery status (in Settings + Payments drawer)

Delivery status	UI
`pending` / `in-flight`	Amber "delivering · attempt {n}/{max}" with next-attempt-at countdown.
`delivered`	Green check + delivered timestamp + attempts used; row shows **net** prominently.
`failed`	Red banner: last HTTP status, last error, "Retry now" button (`POST /v1/deliveries/:id/retry`) — the only manual reconciliation the merchant needs in MVP.
`skipped`	Grey "paid before onboarding — onboard this address to replay it". Not displayed by default.

10 Balances (non-custodial reads)

Because the router is non-custodial, "balance" = the merchant wallet's on-chain balance per accepted token. Read directly with the `quais` SDK — no backend round-trip, no ledger to reconcile in the UI.

Token	Read
Native QUAI	`provider.getBalance(merchantAddress)` → format with 18 decimals.
ERC-20 (mUSDQ, 6 decimals)	`balanceOf(merchantAddress)` on the allowed token address (from `contracts/deployments/<network>.json` → `stablecoin`), ABI-decode, format with token decimals.

Balance card anatomy

QUAI — native

1,284.5678 QUAI

≈ $1,028.00 · rate 0.80 (testnet estimate)

`0x00a3…7f2b` Copy

"Balances update on page focus + every 30 s + after your own sends."

mUSDQ — stablecoin

500.000000 mUSDQ

≈ $500.00 · 1:1 testnet stablecoin

Same copy chip + explorer link. Token list = the contract's allowlist; MVP hardcodes the two deployed tokens and notes "dynamic allowlist later".

Rule	Spec
Units	Always show human units; format with the token's own decimals via `quais` `formatUnits`.
Refresh	On mount, on window `focus`, every 30 s, and immediately after any tx the connected wallet sends. Collapse to one `useBalances()` hook.
Zero state	"Awaiting first payment" empty state with a CTA to Checkout tools — explicit, not a bare 0.
Wrong zone	Show "—" + tooltip "Wallet is on another zone" instead of a misleading zero.

> **MVP: no withdrawal screen**
>
> Payouts are already direct. A "Send" feature is a v2 wallet feature (and the contract needs no involvement); the MVP only shows addresses + copy buttons. Do not build transfer UI into the dashboard.

11 Shared components & proposed file structure

Shared component set (all surfaces use these)

Component	Purpose
`AddressChip`	Truncated checksummed address + copy + popover full form + explorer link (Section 05).
`CopyButton`	The copy primitive with micro-feedback, used by chips, order-id fields, links.
`StatusBadge`	Pending / Paid / Failed / Skipped / Expired mapping to colours + icons.
`AmountText`	Token-aware formatting: human units, decimals, gross/fee/net breakdown, copied from on-chain integers only via `formatUnits`.
`EmptyState`	Icon + title + CTA for zero balances, no payments, no deliveries.
`ConfirmDialog`	Destructive confirmations (cancel order, pause webhooks).

Proposed structure (inside `frontend/`)

```
frontend/
├─ app/
│  ├─ (merchant)/                  # Merchant App — layout w/ sidebar + topbar
│  │  ├─ page.tsx                  # / landing (CTA → onboarding)
│  │  ├─ onboarding/page.tsx       # 4-step wizard (Section 07)
│  │  └─ dashboard/
│  │     ├─ page.tsx               # overview: balances, latest payments, feed, webhook h
│  │     ├─ payments/page.tsx      # payments table + delivery drawer (Section 08)
│  │     ├─ checkout/page.tsx      # order form → QR / payment link (Section 06)
│  │     └─ settings/page.tsx      # profile, webhook URL, active toggle, wallet chip
│  ├─ demo/page.tsx                # QA demo — every component in a real flow (Section 03
│  └─ checkout/[...order]/page.tsx  # hosted checkout page for payment links (reuses modal
├─ src/
│  ├─ checkout/                    # payment-facing UI — no dashboard imports
│  │  ├─ components/               # PaymentButton, PaymentModal, PaymentQR, PaymentStatu
│  │  ├─ lib/                      # uri codec (Section 06), fees math, order-status poll
│  │  └─ index.ts                  # public component API: openCheckout(order) etc.
│  ├─ merchant/                    # dashboard-only: api client, admin proxy, stores
│  │  ├─ api/                      # server actions / route handlers holding ADMIN_API_KE
│  │  └─ stores/                   # wallet store, notification bus, balances hook
│  └─ shared/                      # AddressChip, CopyButton, StatusBadge, AmountText, us
│     └─ wallet/                   # connect card, provider discovery, zone guard (Sectio
├─ config/
│  └─ chains.ts                    # chainId → zone name, explorer base URL, contract add
└─ next.config.ts                  # NEXT_PUBLIC_CONTRACT_ADDRESS, NEXT_PUBLIC_CHAIN_ID
```

Config & env

Var	Value / source

`NEXT_PUBLIC_CHAIN_ID`	`15000` testnet / `9` mainnet (from `contracts/deployments/<network>.json`)
`NEXT_PUBLIC_CONTRACT_ADDRESS`	The UUPS proxy (`payWithQuai` in deployments JSON) — same zone as chain id
`NEXT_PUBLIC_STABLECOIN_ADDRESS`	Allowlisted ERC-20 for the balance cards
`NEXT_PUBLIC_BACKEND_URL`	Relayer API base (orders status, administered routes via server proxy)
`BACKEND_ADMIN_KEY` (server-only)	Never shipped to the client; used by next server actions/route handlers.

12 API contracts the frontend consumes

Backend REST (from `backend/src/api/server.ts`)

Endpoint	Auth	Frontend use
`GET /health`	—	Relayer liveness dot on the dashboard topbar; status card incl. indexer cursor (debug).
`GET /v1/orders/:merchant/:orderId`	public · 60 r/m/IP	`PaymentStatus` polling; Payments-page live state; checkout confirmation. Poll 5–10 s, stop on `settled: true`, honour `429` + `Retry-After`.
`POST /v1/merchants`	admin	Onboarding step 2. 201 → `{ merchantId, address, name, webhookUrl, active, createdAt, webhookSecret }`; the UI never renders `webhookSecret` — server-side concern. 409 on existing address.
`PATCH /v1/merchants/:address`	admin	Settings: `name`, `webhookUrl`, `active` (pause/resume webhooks — "Resume" amber banner on Webhook health strip). Never rotates the secret.
`GET /v1/deliveries`	admin	Notifications poller + Payments table + webhook health (Section 09).
`POST /v1/deliveries/:id/retry`	admin	"Retry now" in the delivery drawer/Settings (Section 09).

Admin routes require `Authorization: Bearer <ADMIN_API_KEY>`. The dashboard calls them only through a server-side proxy (Next route handlers / server actions) so the key never reaches the browser.

Order status response (poll target)

```
GET /v1/orders/0x00…/0x38c1…
200 { "merchant": "0x00…", "orderId": "0x38c1…",
      "token": "0x0000000000000000000000000000000000000000",
      "amount": "25000000", "feeBps": 50, "expiry": "1786480000",
      "settled": true,
      "webhook": { "status": "delivered", "attempts": 1 } }
// 404 = order never registered · settled=false = pending · webhook=null before relayer se
```

Webhook payload shape (what notifications mirror)

```
{ "id": "0x<txHash>:<logIndex>", "type": "payment.confirmed", "created": 1786480000,
  "data": { "merchantId": "mch_ab12…", "merchant": "0x00…", "orderId": "0x…",
            "payer": "0x00…", "token": "0x0…0", "amount": "25000000",
            "feeBps": 50, "fee": "125000", "net": "24875000",
            "txHash": "0x…", "blockNumber": 12345, "timestamp": 1786479990 } }
```

Reconcile against `net`, not `amount`; dedupe on `id` (at-least-once delivery). The frontend only **mirrors** these fields in the payments table and notifications — HMAC signature verification is backend/merchant-side work, not UI.

Contract calls the frontend signs

Call	When
`registerOrder(orderId, token, amount, expiry)`	Merchant wallet signs in Checkout tools; `orderId` = fresh 32-byte hex; `expiry` = 0 → never.
`cancelOrder(orderId)`	Checkout tools "cancel" on an unsettled order (confirm dialog — destructive).
`approve(PayWithQuai, amount)` → `payOrder(merchant, orderId)`	Customer wallet in the payment modal, ERC-20 path. Show the approve step explicitly ("Step 1 of 2 — Allow token").
`payOrderNative(merchant, orderId)`, `value = amount`	Customer wallet, native QUAI path — exact value required.

13 Edge cases & engineering guidelines

Rate limits

- Public order route: 60 requests/min/IP. Never poll faster than every 5 s; exponential backoff from 429 + `Retry-After`.
- Admin list endpoints are local-store reads — still cache with SWR-style stale-while-revalidate, 10 s.

Pending → Paid semantics

- UI: `settled=false` = pending regardless of tx inclusion.
- Only `webhook.status===delivered` flips the paid chip and the bell.
- In-flight retries must auto-refresh (next-attempt countdown), not require a manual refresh.

Address handling

- Checksum via `getAddress()` everywhere before display or copy.
- Never construct routes with raw addresses — encode/validate; 20 bytes hex strictness mirrors the backend.

Reorgs & double-counting

- The relayer guards finality; the UI must not invent additional certainty. A payment may transiently appear pending after a shallow reorg — the poller self-heals.

Server-side secrets

- `ADMIN_API_KEY` lives in server-only route handlers/actions, never in client bundles or env exposed with `NEXT_PUBLIC_`
- Store nothing sensitive in localStorage; secrets are copy-only interactions, never persisted.

Accessibility & states

- Toasts role/status; badges aria-live; modal focus-trap + Escape; contrast on status chips.
- Every async action (register order, onboard, retry) gets loading/error/success states — reuse one `useAsyncAction`.

Time & expiry

- All on-chain times are Unix seconds; convert once near the API boundary.
- Expiry countdowns tick from `Date.now()`, sync on visibility change and focus.

Demo first

- Build `/demo` early; every checkout component is demoed end-to-end (connect → QR → pay → notification) before the dashboard polish phase.
- Testnet rates are "approximate" labels — never present as market data.

Recommended build order (MVP)

#	Milestone	Deliverable
1	Scaffold + shared primitives	`useCopy`, `AddressChip`, `StatusBadge`, `AmountText`, wallet store with zone guard
2	Payment modal, QUAI path	`PaymentModal` end-to-end with native QUAI + `PaymentStatus` polling
3	ERC-20 + approve step & QR	Two-step pay flow, `PaymentQR` + URI codec, payment link + hosted checkout
4	Onboarding wizard	4-step flow (Section 07) + first-payment CTA into Checkout tools
5	Dashboard shell + overview	Sidebar/topbar, balances (Section 10), webhook health, latest payments
6	Payments + notifications	Table, filters, delivery drawer + retry, bell + toasts (Section 09)
7	Settings + polish	PATCH profile, active toggle, empty states, a11y pass, e2e demo run